



TEXAS A&M
UNIVERSITY®

Week 1

Git and GitHub

- Git Cheat Sheet (Modules >> Week 1)

Readings:

- Python Workshop >> Chapter 8: Source Management
- Python Workshop >> Chapter 9: Developing Collaboratively

- Version control is a system that helps developers track and manage changes to their code as it evolves over time. One of the most widely used tools for this is Git.
- Git is a distributed **Version Control System** that enables developers to monitor code history, manage updates efficiently, and collaborate seamlessly with others.

GitHub: Creating a Personal Access Token

<https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/managing-your-personal-access-tokens>

A Personal Access Token:

- Is a credential linked to **your local machine** that grants permission to interact with GitHub repositories.
- It allows you to push updates and perform other Git operations from your local environment
- For this course, we recommend using classic personal access tokens.

- A **repository** is a collection of files related to a project, housed in an isolated workspace where you can make and manage changes.
- Git functions as a silent observer of your directory, automatically detecting modifications to any file and recording their history.
- You can include as many files and folders as needed —**Git will track and version every change automatically.**
- After working on a file locally, you can push your changes to a remote repository to share updates or collaborate with others.

Create a New Repository from Scratch



- Go to **Repositories**. Click the **New** button (green box).
- Enter a repository name and choose a template, if needed. Make sure it's listed under the correct owner.
- Choose the repository visibility:
 - **Public**: Recommended if you want your work to be sharable and reproducible by others.
 - **Private**: Best for confidential or proprietary projects that shouldn't be publicly accessible.
- Add a **README** file to provide an overview and purpose of the project.
- Include a **.gitignore** file to specify which files and folders should be excluded from version tracking. <https://github.com/github/gitignore>
- Select a **license** for your project.
 - Without a license, default copyright laws apply—you retain full rights and others can't reuse or modify your work.
 - If you're building an open-source project, it's strongly encouraged to add a license. Visit <https://choosealicense.com/> to help decide which one fits your needs.

<https://docs.github.com/en/migrations/importing-source-code/using-the-command-line-to-import-source-code/adding-locally-hosted-code-to-github>

The following code with your repo URL will be given once you create the new repo:

1. Open git bash on the local folder
2. `git remote add origin {url}`
3. `git branch -m main`
4. `git push -u origin main`

<https://github.com/haseungyeon3/demo-website>

** Fork it first before you clone your repo

Forking creates a **copy of a repository** on your GitHub account.

Suppose you find an open-source project on GitHub and want to add a new feature

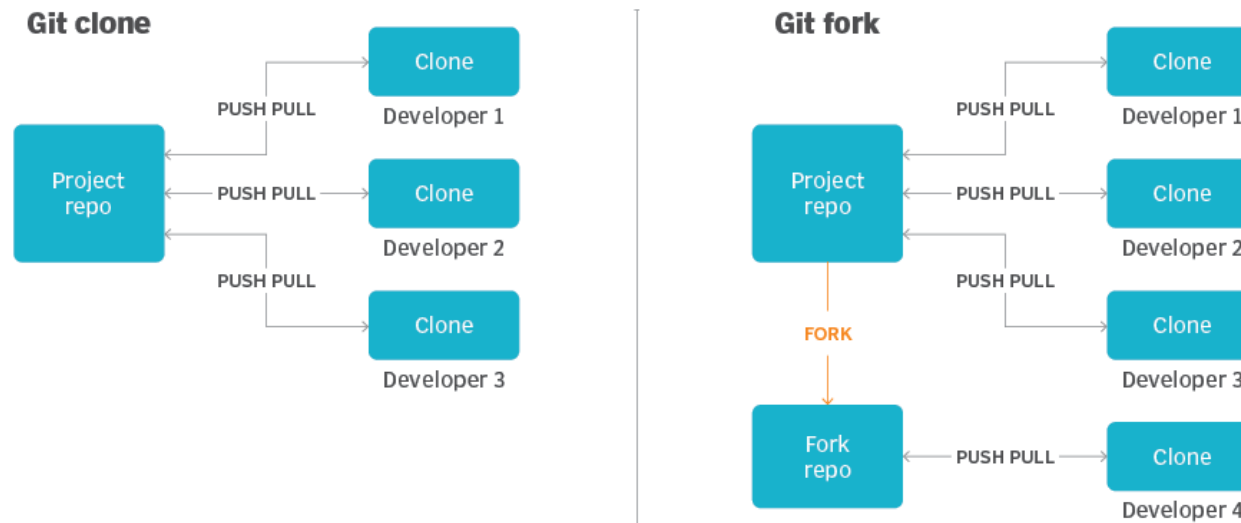
- First create a copy of the repo under your account by **'forking'**
- Then create local repo by **'cloning'**
- After you make changes, **'git push'** them to your forked repo
- Finally, you can submit a **'pull request'** to the owner of original repo to propose your changes

Exercise Continued: Diagram



Git clone vs. fork

Developers who work on a common codebase will clone the repository and then perform push and pull operations to synchronize their changes. In contrast, a fork creates a new codebase and updates to the fork are not synchronized with the original repo.



How to Work on a Repository Locally



To begin working on a project from GitHub (or another Git host) on your own machine, clone the repository using the following command:

```
git clone <URL-of-the-repository>
```

This will create a local copy of the repository that you can edit, commit changes to, and manage with Git.

How to Work on a Repository Locally Continued



To begin working on a project from GitHub (or another Git host) on your own machine, clone the repository using the following command:

```
git clone <URL-of-the-repository>
```

This will create a local copy of the repository that you can edit, commit changes to, and manage with Git.

A **commit object** represents a snapshot in the history of a repository. Every time you run **git commit**, a new commit is created.

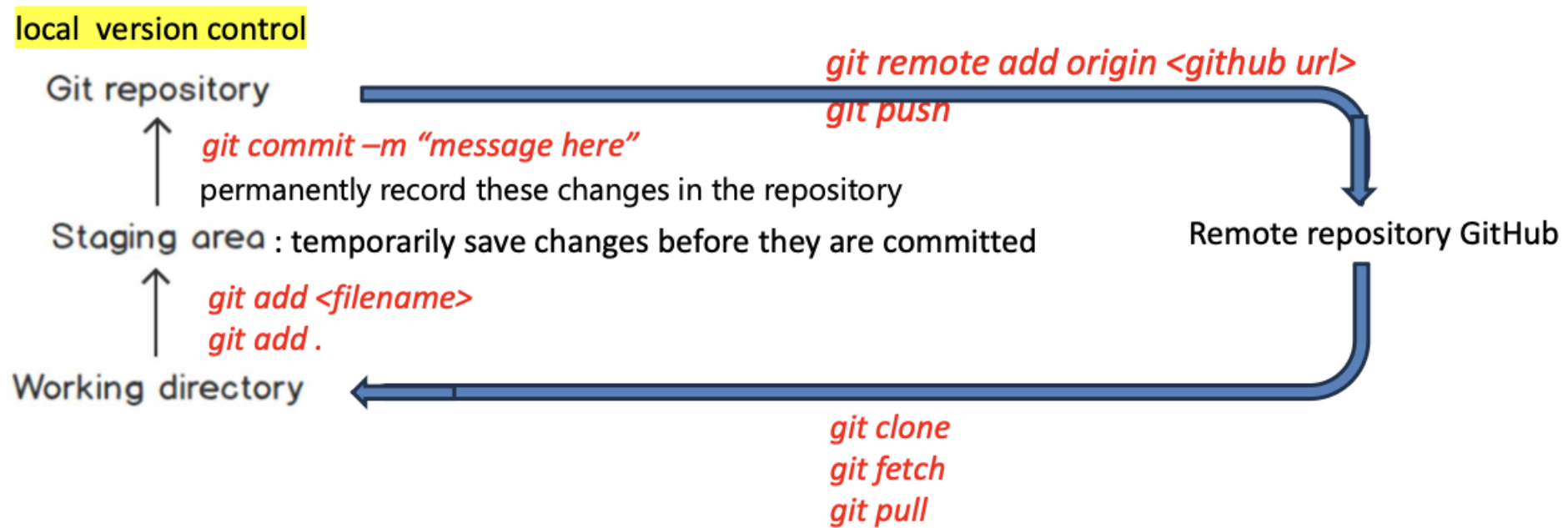
Each commit includes a title and body (the message), the author, timestamps for the changes, and a unique identifier known as the **SHA** or **hash**—which is crucial for referencing specific entries.

To **view commit history**, you can:

- Visit <https://github.com/<your-repo-name>/commits>
- Or use the command **git log** in your terminal to see a detailed list of past commits

Commit and Push

You don't need to do this every time: you can make the changes on your local machine and push the changes to remote repo at a specific checkpoint.



Staging Area and Git Repo

When you are working locally and making changes, git will report that those changes happened, and they are not saved.

git status to see what files were modified.

git add to save those changes in the **staging area** in preparation for a commit. It can be used in files or folders to add all the files within that folder.

Once they are added to the staging area, the next **git commit** command will save the changes in the repository through a commit object.

git diff to see all the changes that have been made locally but are not yet part of the staging area or a commit.

Sometimes, you don't want to add all the contents of a file to the staging area, just part of them. Both ***git commit*** and ***git add*** have an option to guide you through the different changes in the file and allow you to select which ones you want to add.

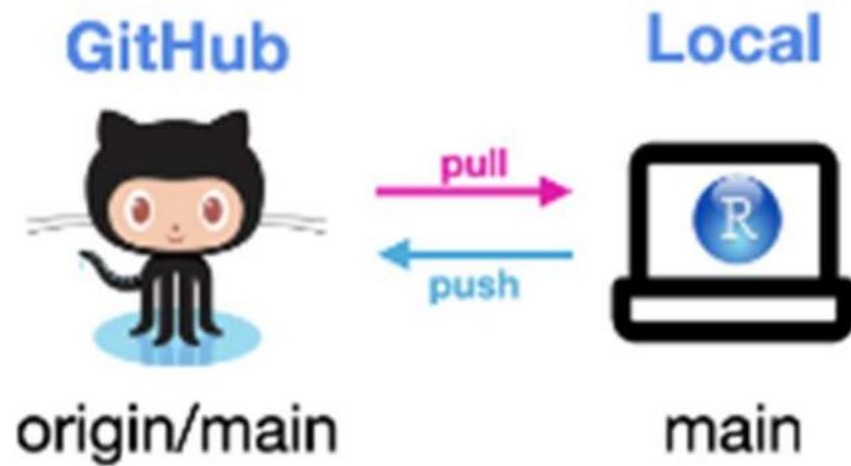
This is through the ***-p*** option, which will ask you for each of the changed chunks within your code, which ones you do want to add.

Undoing Local Changes when you are working on a file:

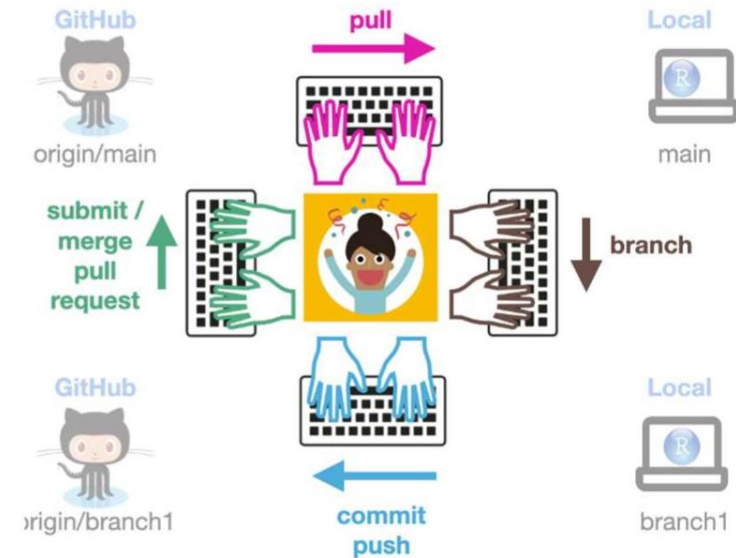
- ***git checkout*** to undo our changes and come back to the version you have saved in the staging area or in the last commit;
- ***git reset*** to revert our repository back to a previous commit in history

Developing Collaboratively: Workflow

Read Chapter 9: Developing Collaboratively



The no branch workflow
- Just you, no collaborator



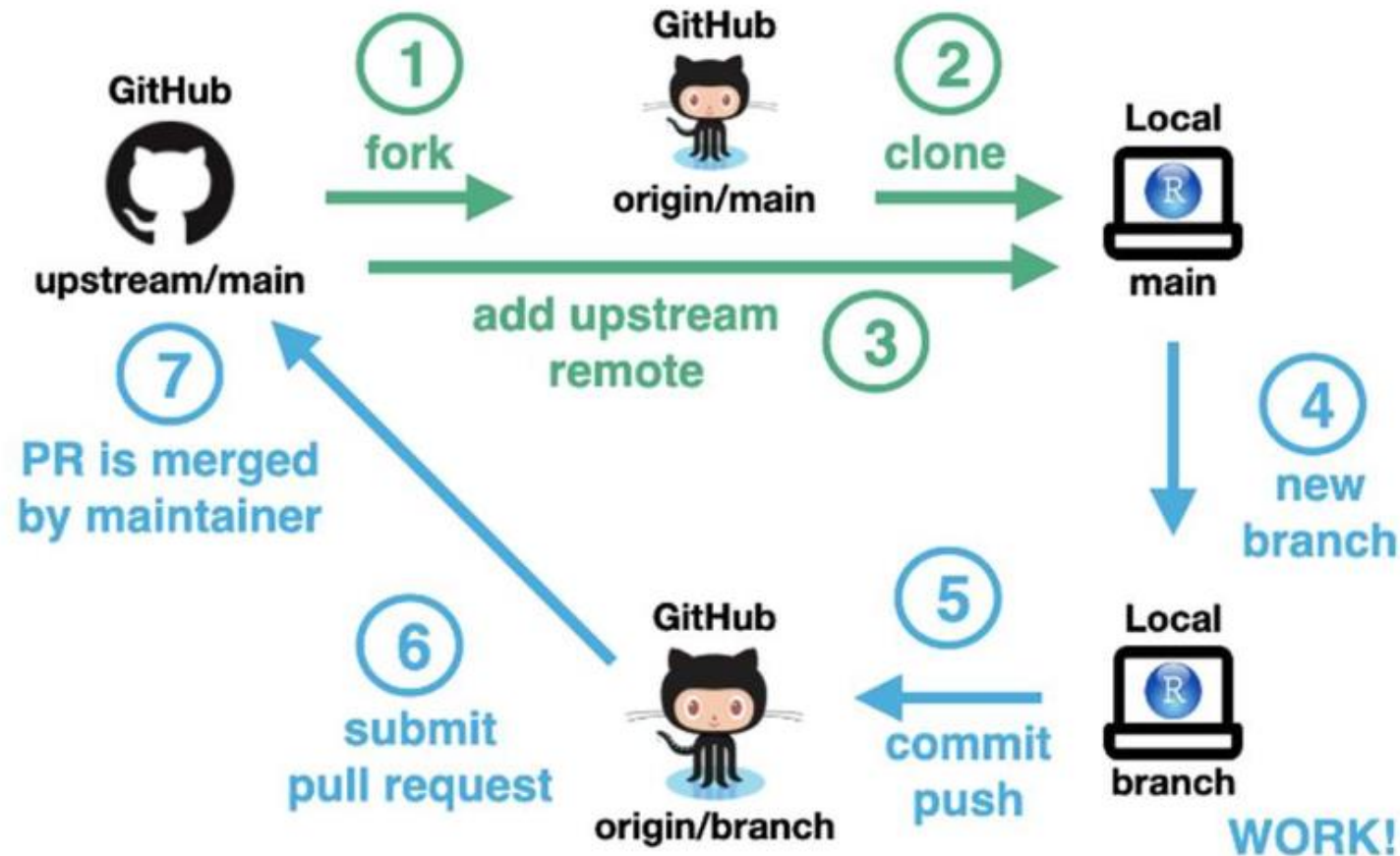
Your repo with branching

Developing Collaboratively: First Pull Request with Branching



TEXAS A&M
UNIVERSITY

1st Pull Request on Another Repo with Branching



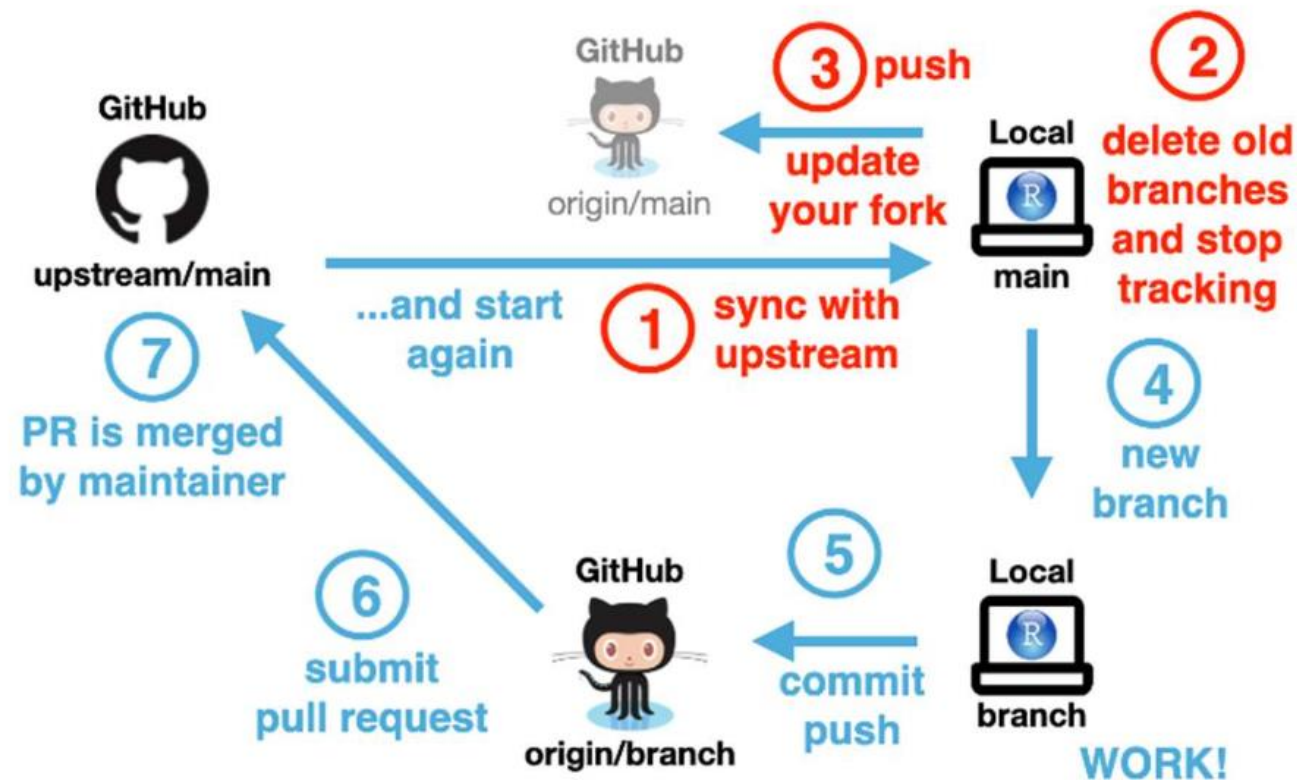
Developing Collaboratively: First Pull Request with Branching Continued



TEXAS A&M
UNIVERSITY

2nd-nth Pull Request on Another Repo with Branching

Syncing a Fork: <https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/working-with-forks/syncing-a-fork>



Your repository currently doesn't exist on your local machine. What is the appropriate steps to comprise Git pull request workflow?

1. Clone the remote GitHub repo to your local machine
2. Create and checkout a new branch in your local repo
3. Commit the file changes
4. Push the staged change to the GitHub repo
5. Open a pull request to merge the changes to main branch.

Creating a new Git branch:

- To work on a specific feature, bug fix, or experiment without affecting the main or stable codebase.
- To isolate changes, allowing you to develop, test, and review code independently.

The main branch is what's created by default in a repository, from which people usually clone and pull code. A new development branch is created to protect the origin/main branch from contamination, providing a space where I have the freedom to make any changes.

```
git branch <branchname>
```

```
git checkout -b <branchname>
```

A **pull request** facilitates code review, discussion, and merging of changes in a collaborative environment.

The git command '**git pull**' does **not** perform the same function as a **pull request**! It is used to fetch and merge changes from a remote repo to your local repo. A pull request which allows you to propose/merge changes on remote repo, must be created and merged on GitHub.

Resolving a merge conflict on GitHub:

<https://docs.github.com/en/pull-requests/how-tos/merge-and-close-pull-requests/resolving-a-merge-conflict-on-github>

Basic Procedure – branching to merging PR



1. Start from the latest target branch:

```
git checkout main  
git pull origin main
```

2. Create a branch for changes:

```
git branch <branchname>  
git checkout -b <branchname>
```

3. Make your code changes, then check what changed:

```
git status  
git diff
```

4. Stage and commit the changes:

```
git add .  
git commit -m "your comment here"
```

5. Push your branch to GitHub:

```
git push -u origin <branchname>
```

6. On GitHub, open the repository and click Compare & pull request. Confirm:

Base = branch receiving the change, usually main

Compare = your feature branch

7. Write a clear title and short description, then click **Create pull request**. (<https://docs.github.com/en/pull-requests/how-to/create-pull-requests/creating-a-pull-request>)

8. After opening PR

- Request a review from a teammate.
- Respond to comments and push fixes to the **same branch**; GitHub automatically adds those commits to the existing PR.
- Once approved and checks pass, click **Merge pull request** (if you have permission), then delete the feature branch.